

HiDiTingV100 TLS&DTLS

开发指南

文档版本	00B01
发布日期	2025-06-05

前言

概述

本文档主要介绍 TLS/DTLS 组件的开发实现示例。

TLS/DTLS 以及其他加密套基于开源组件 mbedtls 3.1.0 实现，详细说明请参见官方说明：<https://tls.mbed.org/api/index.html>。

如果官方说明版本与 SDK 版本不一致，请参考官方 release 说明：

<https://github.com/ARMmbed/mbedtls/releases>。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....	i
1 API 接口说明.....	1
1.1 结构体说明	1
1.2 API 列表	1
1.3 配置说明.....	1
2 开发指南.....	2
3 硬化适配.....	3
3.1 硬化适配范围	3
3.2 硬化 API	4
3.3 适配说明.....	6
3.3.1 AES 适配.....	6
3.3.2 SHA256 适配	6
3.3.3 大数模幂适配	6
3.3.4 随机数适配.....	6
3.3.5 CCM 适配.....	7
3.3.6 RSA 数字签名适配.....	7
4 注意事项.....	8
4.1 关于 MBEDTLS_SHA256_ALT 的注意事项	8
4.2 关于配置 SSL 接收缓存的注意事项	9
4.3 随机数使用注意事项.....	10
4.4 对称加密使用注意事项.....	10
4.5 非对称加密使用注意事项	11
4.6 哈希函数使用注意事项.....	12

4.7 消息认证码使用注意事项	12
4.8 安全协议使用注意事项	13

Draft

1 API 接口说明

1.1 结构体说明

1.2 API 列表

1.3 配置说明

1.1 结构体说明

MBEDTLS 详细的结构体说明请参考官方说明文档：<https://fossies.org/dox/mbedtls-3.1.0/annotated.html>

1.2 API 列表

MBEDTLS 详细的 API 说明请参考官方说明文档：<https://fossies.org/dox/mbedtls-3.1.0/files.html>

1.3 配置说明

MBEDTLS 详细的配置项说明请参考官方说明文档：https://fossies.org/dox/mbedtls-3.1.0/mbedtls__config_8h.html#a54a11655ee76cbd7b4d8b159e377a7def

2 开发指南

mbedtls 详细的开发 DEMO 请参考官方说明文档: https://fossies.org/dox/mbedtls-3.1.0/md_docs_getting_started.html

Draft

3 硬化适配

- 3.1 硬化适配范围
- 3.2 硬化 API
- 3.3 适配说明

3.1 硬化适配范围

范围	安全算法	硬化适配范围	编译宏	对外影响
对称加密	AES	AES-ECB/CBC/CTR	MBEDTLS_AES_ALT	不支持 AES-192。
	GCM	AES-GCM	MBEDTLS_GCM_ALT	除 AES 之外，其他对称加密算法的 GCM 模式不再支持。AES 仅支持 AES-128-GCM 与 AES-256-GCM。
	CCM	AES-CCM	MBEDTLS_CCM_ALT	除 AES 之外，其他对称加密算法的 GCM 模式不再支持。AES 仅支持 AES-128-CCM 与 AES-256-CCM。
非对称加密	RSA	V15-Encrypt V21-Encrypt V15-Decrypt V21-Decrypt	MBEDTLS_RSA_ENCRYPT_ALT MBEDTLS_RSA_DECRYPT_ALT	不支持 2048bit 以下的 RSA 加解密。

范围	安全算法	硬化适配范围	编译宏	对外影响
		V15-Sign V21-Sign V15-Verify V21-Verify	MBEDTLS_RSA_SIGN_ALT MBEDTLS_RSA_VERIFY_ALT	不支持 2048bit 以下的 RSA 签名与验签。
哈希	SHA256	Start-Update-Finish Clone	MBEDTLS_SHA256_ALT	SHA256 update 或者 clone 操作时会分配硬件资源，如果硬件资源耗尽而无法分配，则相关计算失败。
大数	大数模幂	大数模幂	MBEDTLS_MPI_EXP_MOD_ALT	不支持大于 4096bit 的大数模幂。
随机数	熵源	平台随机数	MBEDTLS_ENTROPY_HARDWARE_ALT	将硬件随机数发生器注册为 mbedTLS 的强安全熵源。用户不允许关闭该宏。

3.2 硬化 API

范围	API	头文件	对外影响
AES	All APIs in AES	mbedtls/aes.h	• mbedtls_aes_context 结构体定义发生改变。 • 不支持 AES-192。
CCM	All APIs in CCM	mbedtls/ccm.h	• mbedtls_ccm_context 结构体定义发生改变。 • 仅支持 AES-128/256-CCM，其他对称加密算法的 CCM 模式不再支持。
GCM	All APIs in GCM	mbedtls/gcm.h	• mbedtls_gcm_context 结构体定义发生改变。 • 仅支持 AES-128/256-GCM，其他

范围	API	头文件	对外影响
			对称加密算法的 GCM 模式不再支持。
SHA256	All APIs in SHA256	mbdtdls/sha256.h	- mbdtdls_sha256_context 结构体发生改变。 - SHA256clone 可能失败。 - SHA256 internal 接口不再支持。
大数模幂	mbdtdls_mpi_exp_mod	mbdtdls/bignum.h	4096 位及以下，使用硬件完成大数模幂计算，4096 位以上不支持，返回错误码。
Random	mbdtdls_hardware_poll	mbdtdls/entropy_poll.h	通过 HRNG 获取安全随机数。
RSA	mbdtdls_rsa_rsaes_pkcs1_v15_encrypt	mbdtdls/rsa.h	RSA V15 加密使用硬件完成。
	mbdtdls_rsa_rsaes_oaep_encrypt	mbdtdls/rsa.h	RSA V21 加密使用硬件完成。
	mbdtdls_rsa_rsaes_pkcs1_v15_decrypt	mbdtdls/rsa.h	RSA V15 解密使用硬件完成。
	mbdtdls_rsa_rsaes_oaep_decrypt	mbdtdls/rsa.h	RSA V21 解密使用硬件完成。
	mbdtdls_rsa_rsassa_pkcs1_v15_sign	mbdtdls/rsa.h	RSA V15 签名使用硬件完成。
	mbdtdls_rsa_rsassa_pss_sign	mbdtdls/rsa.h	RSA V21 签名使用硬件完成。
	mbdtdls_rsa_rsassa_pkcs1_v15_verify	mbdtdls/rsa.h	RSA V15 验签使用硬件完成。
	mbdtdls_rsa_rsassa_pss_verify	mbdtdls/rsa.h	RSA V21 验签使用硬件完成。

3.3 适配说明

3.3.1 AES 适配

- 使能 MBEDTLS_AES_ALT 后，ECB 和 CBC 模式的 AES 算法会直接调用硬件驱动接口，而其他加密模式会沿用软件逻辑，最终调用硬件提供的 AES_ECB 算法完成加速。
- 使能 MBEDTLS_AES_ALT 后，AES 算法在使用硬件加速器时，会锁定硬件加速器资源，即 AES 操作是阻塞的，直至驱动获取资源或超时返回失败。

3.3.2 SHA256 适配

- 使能 MBEDTLS_SHA256_ALT 后，SHA256 将使用硬件驱动接口，将不再支持 SHA224 操作。
- 使能 MBEDTLS_SHA256_ALT 后，SHA256 算法在使用硬件加速器时，会锁定硬件加速器资源，即 SHA256 操作是阻塞的，直至驱动获取资源或超时返回失败。

3.3.3 大数模幂适配

- 发布版本中默认已对大数模幂操作（API 接口为：mbedtls_mpi_exp_mod）完成硬件适配。目前仅支持 4096bit（含）以下的大数模幂操作，超过 4096bit 时，大数模幂操作失败，返回值为 MBEDTLS_ERR_MPI_BAD_INPUT_DATA。
- 硬件加速大数模幂操作时，会根据大数参数的位数，额外从堆中分配 1KB ~ 4KB 的内存，如果分配失败，大数模幂操作会失败，返回值为 MBEDTLS_ERR_MPI_ALLOC_FAILED。

3.3.4 随机数适配

使能 MBEDTLS_ENTROPY_HARDWARE_ALT 后，系统会选用默认增加硬件随机数作为一个强随机数源，用户仍然可以增加额外的随机数源。

MBEDTLS_ENTROPY_HARDWARE_ALT 编译宏在发布版本中是默认开启的，除非用户明确已经注册其他强随机数源，否则此编译宏不允许被关闭。如果此宏被关闭，而用户也没有注册其他强随机数源，会导致 mbedtls 无法提供安全随机数，严重影响系统的安全性。

3.3.5 CCM 适配

对于加密方式为 AES，密钥长度为 128bit 的 CCM 的加密操作和解密操作，mbedTLS 已实现硬件适配，可以提升 CCM 加密与解密的速度。

- CCM 加密的 API 接口为：mbedtls_ccm_encrypt_and_tag。
- CCM 解密的 API 接口为：mbedtls_ccm_auth_decrypt。

3.3.6 RSA 数字签名适配

config.h 文件中使能 MBEDTLS_RSA_HW_ACCEL_BY_CIPHER 编译宏之后，mbedTLS 会对密钥长度为 2048bit 与 4096bit 的 RSA 数字签名操作和验签操作进行硬件加速。对于密钥长度为非 2048bit 或者非 4096bit 的 RSA 数字签名与验签，仍然使用软件完成。

- RSA 数字签名操作的 API 接口为：mbedtls_rsa_pkcs1_sign。
- RSA 数字验签操作的 API 接口为：mbedtls_rsa_pkcs1_verify。

MBEDTLS_RSA_HW_ACCEL_BY_CIPHER 编译宏在发布版本中是默认开启的，如果用户希望节省代码体积，可以关闭此宏，但是关闭后会降低 2048bit 与 4096bit 的 RSA 数字签名操作和验签操作的速度。

4 注意事项

- 4.1 关于 MBEDTLS_SHA256_ALT 的注意事项
- 4.2 关于配置 SSL 接收缓存的注意事项
- 4.3 随机数使用注意事项
- 4.4 对称加密使用注意事项
- 4.5 非对称加密使用注意事项
- 4.6 哈希函数使用注意事项
- 4.7 消息认证码使用注意事项
- 4.8 安全协议使用注意事项

4.1 关于 MBEDTLS_SHA256_ALT 的注意事项

使能 MBEDTLS_SHA256_ALT 后，在使用时需要注意以下细节：

- SHA256 HASH 或 HMAC 操作不能嵌套，即必须每次执行完当前 SHA256 HASH/HMAC 操作后，再进行下一次操作。

例如：以下操作在 ctxA 计算完成前，就执行 ctxB 计算，ctxA 会在下方步骤 7 开始返回失败，并且无法得到预期的结果。

```
mbbedtls_sha256_context *ctxA, *ctxB;  
unsigned char *inputA1, *inputA2, *inputB1, *inputB2, *outputA, *outputB;  
...  
1: mbedtls_sha256_starts_ret(&ctxA);  
2: mbedtls_sha256_update_ret(inputA1, 64);
```

```
3: mbedtls_sha256_starts_ret(&ctxB);
4: mbedtls_sha256_update_ret(inputB1, 64);
5: mbedtls_sha256_update_ret(inputB2, 64);
6: mbedtls_sha256_finish_ret(outputB, 32);
7: mbedtls_sha256_update_ret(inputA2, 64);
8: mbedtls_sha256_finish_ret(outputA, 32);
```

- 同时使能 MBEDTLS_SHA256_ALT 和 MBEDTLS_AES_ALT 时, SHA256 HASH 或 HMAC 操作不能和 AES 操作嵌套, 即必须每次执行完当前 SHA256 HASH/HMAC 操作后, 再进行 AES 操作。

例如: 以下操作在 ctxA 计算完成前, 就执行 AES 计算, ctxA 会在下方步骤 5 开始返回失败, 并且无法得到预期的结果。

```
mbedtls_sha256_context *ctxA;
unsigned char *inputA1, *inputA2, *outputA, *inputDec, *outputDec;
mbedtls_aes_context *ctxDec = aes_decrypt_init(aesKey, 16);
...
1: mbedtls_sha256_starts_ret(&ctxA);
2: mbedtls_sha256_update_ret(inputA1, 64);
3: aes_decrypt(ctxDec, inputDec, outputDec);
4: aes_decrypt_deinit(ctxDec);
5: mbedtls_sha256_update_ret(inputA2, 64);
6: mbedtls_sha256_finish_ret(outputA, 32);
```

- 使能 MBEDTLS_SHA256_ALT 后, 将不支持 SHA224, 使用 SHA224 时会返回失败。

4.2 关于配置 SSL 接收缓存的注意事项

- SSL 接收缓存由编译项 MBEDTLS_SSL_IN_CONTENT_LEN 控制, 默认为 16KB。如果实际应用中, 用户可以确保 SSL 上层数据包的最大长度不超过 2KB 或 4KB, 则可以通过 mbedtls_ssl_conf_max_frag_len 接口设置 SSL 接收缓存的长度, 达到节省内存的目的。

📖 说明

mbedtls_ssl_conf_max_frag_len 接口的调用必须先于 mbedtls_ssl_setup 接口。

- 考虑到多级数字证书可能导致 TLS 握手包的长度大于 1KB，因此调用 `mbedtls_ssl_conf_max_frag_len` 接口时，只有 `mfl_code` 为 `MBEDTLS_SSL_MAX_FRAG_LEN_2048` 或 `MBEDTLS_SSL_MAX_FRAG_LEN_4096` 时，`mbedtls` 才会修改接收缓存；如果 `mfl_code` 为 `MBEDTLS_SSL_MAX_FRAG_LEN_512` 或 `MBEDTLS_SSL_MAX_FRAG_LEN_1024`，则接收缓存仍然为 16KB。
- 如果修改 SSL 接收缓存为 2KB 或 4KB 后，Client 端接收到 Server 端的一个大于 2KB 或 4KB 的数据包，此时 `mbedtls_ssl_read` 接口会返回失败，错误码为 `MBEDTLS_ERR_SSL_MSG_TOO_LONG`（此为新增的一个特定错误码），当用户获得此错误码时，必须关闭 SSL 连接，不允许继续从 SSL 链路接收数据。
- 通过 `mbedtls_ssl_conf_max_frag_len` 接口设置 SSL 接收缓存，目前只对 SSL Client 有效。

4.3 随机数使用注意事项

- LiteOS 系统没有类似 Linux 的 `/dev/random` 随机设备，用于产生安全随机数，因此 `mbedTLS` 的编译宏 `MBEDTLS_NO_PLATFORM_ENTROPY` 必须开启。
- 密码算法中使用的随机数必须是安全随机数。`mbedTLS` 发布版本中已经将芯片的硬件随机数发生器注册为唯一的 strong 熵源，用户通过 `mbedtls_entropy_func` 函数获取的随机数即为安全随机数。禁止将非安全随机数生成器注册为 `mbedTLS` 的 strong 熵源。

4.4 对称加密使用注意事项

- 禁止使用不安全的对称加密算法，包括 DES。`mbedTLS` 发布版本中默认关闭这些不安全算法，出于安全性考虑，用户禁止再开启这些算法。
- 使用分组密码算法时，应优先选择 GCM 模式，GCM 模式是目前业界推荐使用的对称加密算法工作模式。
- 组密码算法使用到的 IV 值，必须满足具体加密模式对 IV 值的要求。对于 CBC/CFB 模式，IV 在产生之前都不应该被预测出来；对于 OFB/GCM/CCM 模式，只需保证 IV 的唯一性；对于 CTR 模式，需保证各 COUNTER 的唯一性。唯一性是指 对于相同的密钥，每次加密数据的 IV 各不相同。分组加密算法的 IV 无机密性要求，存储或传输时无需进行加密保护。

- 流密码算法须结合消息认证码校验消息完整性。单纯的流加密场景下，推荐使用 AES-OFB/CTR 或者 chacha20。流加密算法无法防止比特反转攻击，在使用上述流加密算法时，如在加密的同时须保证被加密数据的完整性，产品可以额外计算消息认证码（MAC）来验证被加密数据的完整性，也可以使用本身含完整性校验机制的流加密算法，如使用 AES-GCM、AES-CCM 或 chacha20-poly1305 这些认证加密算法。
- 使用分组加密算法时，填充方式建议选择 CMS-Padding 或 ISO-Padding，也就是调用 mbedTLS 的 `mbedtls_cipher_set_padding_mode` 函数时，`mode` 应该等于 `MBEDTLS_PADDING_PKCS7` 或者 `MBEDTLS_PADDING_ONE_AND_ZEROS`，LiteOS mbedTLS 发布版本中已默认关闭另外两种填充方式（`MBEDTLS_CIPHER_PADDING_ZEROS_AND_LEN` 与 `MBEDTLS_CIPHER_PADDING_ZEROS`），出于安全性考虑，用户禁止再打开。

4.5 非对称加密使用注意事项

- 非对称密码的安全强度应该大于等于 128bit，因此要求 RSA 算法的密钥长度应大于等于 3072bit，ECDSA/ECDH 的密钥长度应大于等于 256bit。2048bitRSA 密钥长度可以继承使用，但是用户应尽快切换至 3072bit。
- 使用 RSA 算法时要选取合适的公共指数 e ，mbedTLS 代码中对于公共指数 e 的限制是不能小于 3，实际使用中 e 推荐取值至少为 $2^{16} + 1$ ，即 0x10001。
- 使用 RSA 算法进行加密操作或者签名操作时，禁止 PKCS#1_V15 填充，应使用 PKCS#1_V21 填充方式。mbedTLS 支持 V15 与 V21 两种填充方式，对应的编译宏分别是 `MBEDTLS_PKCS1_V15` 与 `MBEDTLS_PKCS1_V21`，并且 LiteOS mbedTLS 发布版本中都默认开启。实际使用中用户应该选择 V21 填充方式，也就是调用 `mbedtls_rsa_set_padding` 函数时，`padding` 应该等于 `MBEDTLS_RSA_PKCS_V21`。发布版本中默认开启 `MBEDTLS_PKCS1_V15` 的原因是目前许多证书使用的仍然是 V15 填充方式，用户应认识到 V15 填充方式的风险，并尽快切换至 V21 填充。
- 使用非对称算法时，加密和签名要使用不同的密钥对。签名密钥对的私钥仅用于签名，其对应的公钥用于验证签名，而加密密钥对的公钥仅用于加密，其对应的私钥用于解密。
- 在同时需要对数据进行对称加密和数字签名时，使用先签名后加密的方式。先签名后加密是指先对消息进行签名，然后对消息的签名值和消息一起进行加密。如果采用先加密后签名的方式，接收方只能知道该消息是由签名者发送过来的，但不能确定签名者是否是该消息的创建者。

- 在使用 DH/ECDH 进行密钥协商时支持完全前向保密特性。在 TLS/DTLS 协议中，如使用 DH/ECDH 算法进行密钥协商，选取包含 DHE 或 ECDHE 密钥交换算法的加密套件即可支持完全前向保密特性，应避免选取仅包含 DH/ECDH 的加密套件。mbedTLS 发布版本中已默认关闭所有 DH/ECDH 密钥协商类加密套件，仅开启 DHE/ECDHE 加密套件。
- 推荐使用业界公认安全的椭圆曲线，避免使用 NIST 定义的椭圆曲线。NIST 定义的椭圆曲线域参数由一个固定种子产生，由于 NIST 并未解释该种子的来源，因此不建议选择 NIST 曲线。mbedTLS 发布版本中，默认开启了四条 ECC 曲线，分别是 SECP256R1, SECP384R1, BP256R1, Curve25519，其中 SECP256R1/SECP384R1 属于 NIST 曲线，SECP256R1 曲线默认使能的原因是 WiFi 与 BLE 协议中会使用，SECP384R1 曲线默认使能的原因是鸿蒙 app 校验业务中会使用。用户自己的安全方案中应避免使用 SECP256R1、SECP384R1 这类曲线，可以选择 BP256R1 曲线。如果 BP256R1 曲线的性能不满足要求的话，可以选择打开 SECP256K1 曲线，这条曲线的性能与 SECP256R1 相等，代价是会增大一定的代码体积。

4.6 哈希函数使用注意事项

- 禁止使用不安全的哈希算法，包括 MD5。
- 使用 PBKDF2 密钥派生算法对口令等敏感数据做单向哈希时，应该对 PBKDF2 的各项参数进行限制，包括如下几条安全性要求（mbedTLS 代码层面未对这些要求做限制，需要用户使用时保证）：
 - 迭代次数默认推荐至少 10000 次。如果平台支持 PBKDF2 硬化，可适当增加迭代次数，提升安全性。
 - 哈希函数推荐选择 SHA256 或更安全的哈希算法。
 - 盐值至少 16Byte，应使用安全的随机数。
 - 用于口令单向哈希时，哈希算法输出长度应该不小于 256bit。

4.7 消息认证码使用注意事项

- 禁止在密码协议中使用截短的消息认证码。
- 禁止将密钥与消息进行拼接后进行哈希运算以达到计算 MAC，推荐使用 HMAC 或者 CMAC。

- 同一密钥不允许既用于计算加解密，又用于计算消息认证码 MAC。可参考 TLS/DTLS 协议对于数据的加解密与消息认证码的处理方式。

4.8 安全协议使用注意事项

- 禁止使用 SSL3.0, TLS1.0, TLS1.1 协议。mbedtls3.1.0 已不再支持这三个协议。
- TLS 协议中禁止使用 CBC 模式的加密套件。该类套件在密码协议中使用常常会因为实现不当引入漏洞而被攻击者利用分析明文，并且已被 HTTP2 列入黑名单。由于 CBC 模式的加密套件仍在被大量使用，因此 LiteOS mbedtls 发布版本中默认并未关闭 CBC 模式的加密套件，但是我们新增加了一个编译宏 MBEDTLS_REMOVE_CBC_CIPHERSUITES，此编译宏默认关闭，如果用户确定当前不会使用 CBC 模式的加密套件，应该开启此编译宏。
- TLS/DTLS 安全协议握手阶段，对于使用 PKI 证书进行身份认证的加密套件，必须对接收到的 X.509 证书的有效时间进行检查。mbedtls 定义了一个编译宏 MBEDTLS_HAVE_TIME_DATE，关闭此编译宏后 mbedtls 不再对 X.509 证书的有效时间进行检查，因此出于安全性考虑，用户禁止关闭此宏。另外，用户必须保证系统时间的正确性，才能准确的校验 X.509 证书的有效时间，防止过期证书错误的通过校验。